

Adapting OpenAI Privacy Filter for Turkish PII Detection

Final Project Report — Natural Language Processing

Author: Boran Toktay

Project Track: Track 2 — Enhancement of an Existing Project

Base Model: openai/privacy-filter

Dataset: BTX24/turkish-privacy-pii-ner

Fine-tuned Model: BTX24/turkish-privacy-filter-pii

Abstract

This project adapts OpenAI Privacy Filter, an existing privacy-oriented token classification model, to Turkish personally identifiable information (PII) detection. The work follows Track 2 of the final project specification because it starts from an existing open-source/open-weight project and implements a meaningful language-specific enhancement. The main contribution is a Turkish privacy span detection pipeline covering ten privacy labels: tckn, secret, iban, vkn, account_number, private_address, private_date, private_phone, private_email, and private_person. A fully synthetic Turkish Privacy PII NER dataset was generated, validated, split, uploaded to Hugging Face, and used to fine-tune the model. The final dataset contains 103,923 JSONL rows and 7.4 million characters. Fine-tuning was performed in Google Colab Pro using a GPU runtime, and the best checkpoint was selected at epoch 3 with a validation loss of 0.00216. On the synthetic test split, the fine-tuned model achieved 0.9997 detection F1 and 0.9983 span F1. Although these results are very strong for synthetic Turkish PII, the report also discusses limitations, observed error cases, and the need for future evaluation on real or manually curated Turkish text before production use.

1. Introduction

Personally identifiable information detection is a practical NLP problem for privacy-preserving AI systems. Modern AI applications often process user messages, logs, forms, support tickets, educational records, health-related notes, and model training data. If sensitive fields such as names, identity numbers, phone numbers, e-mails, IBANs, addresses, or tokens are not detected and masked, they can leak into logs, model prompts, datasets, or downstream analytics pipelines.

OpenAI Privacy Filter provides a strong starting point for this problem because it is designed for PII span detection and redaction. However, the released model is primarily English-oriented. Turkish introduces additional challenges: agglutinative suffixes attach to names, local identifiers such as TCKN and VKN are not native English PII categories, Turkish address structure is different, and Turkish phone/IBAN formats require localized examples. Therefore, the goal of this project is to adapt the Privacy Filter approach to Turkish by constructing a Turkish privacy dataset and fine-tuning the base model on that dataset.

The final project deliverable includes a published Hugging Face dataset, a fine-tuned Hugging Face model, a Colab training notebook, and this report. The project was completed by Boran Toktay.

2. Project Objective and Research Questions

The objective is to build a reproducible Turkish PII detection pipeline based on OpenAI Privacy Filter. The model is not intended to be a general chatbot, a text classifier, or an instruction-tuned system. It is a span-based token classification model: the expected output is the label and character span of each sensitive entity in a Turkish text.

The project is guided by three research questions:

- RQ1: Can an English-oriented privacy span detector be adapted to Turkish through supervised fine-tuning?
- RQ2: Which Turkish privacy labels are easiest or hardest to learn from synthetic data?
- RQ3: What implementation and modeling problems appear when adapting Privacy Filter to a custom Turkish label space?

3. Label Space

A custom Turkish privacy label space was defined. The label order follows the Privacy Filter custom label-space requirement that the outside class O appears first. The final span_class_names configuration is shown below.

```
{
  "category_version": "tr_privacy_v1",
  "span_class_names": [
    "O",
    "tckn",
    "secret",
    "iban",
    "vkn",
    "account_number",
    "private_address",
    "private_date",
    "private_phone",
    "private_email",
    "private_person"
  ]
}
```

Label	Meaning in this project
tckn	Synthetic Turkish national identity-like values.
secret	Synthetic passwords, OTP codes, API-key-like strings, tokens, recovery codes, and session/reset tokens.
iban	Synthetic Turkish IBAN-like values.
vkn	Synthetic Turkish tax identification-like values.
account_number	Customer numbers, account IDs, reference codes, membership IDs, ticket/order references, and card last-four examples.
private_address	Synthetic Turkish-style address expressions.
private_date	Privacy-relevant dates such as birth dates, registration dates, appointment dates, or application dates.
private_phone	Turkish mobile-number-like synthetic phone numbers.
private_email	Synthetic non-routable e-mail addresses.
private_person	Synthetic Turkish-style person names, including suffix and honorific contexts.

4. Dataset Construction

The largest part of the project was dataset construction. Existing Turkish NER resources are useful for general person and date recognition, but they do not fully cover privacy-specific Turkish fields such as TCKN, VKN, IBAN, structured account identifiers, OTPs, API-key-like strings, or non-routable e-mail patterns. For this reason, the project used a synthetic data strategy rather than relying only on existing NER corpora.

Synthetic examples were generated with the help of multiple AI systems, including Gemini, DeepSeek, and Grok. These tools were used mainly to create natural Turkish sentence variations across realistic contexts such as hospital registration, school forms, cargo support, CRM notes, bank refunds, invoice processing, technical

support, and user account workflows. The sensitive values themselves were designed to be fake, non-routable, or mock values. For example, e-mail domains used .invalid domains, TCKN and VKN examples were mock values, and IBAN-like strings were synthetic.

A key design choice was to avoid asking AI models to create trusted labels directly. Instead, the generation process produced text plus known entity values. Character-level start and end offsets were then created and validated programmatically. This reduced annotation drift and made the dataset suitable for span-based NER rather than simple text classification.

4.1 Dataset Statistics

Metric	Value
Total JSONL rows	103,923
Label classes	10 privacy labels
Total CSV source files	105
JSONL file size	20 MB
CSV file size	10 MB
Total characters	7,404,532
Average text length	71.3 characters
Average entity length	19.2 characters
Minimum entity length	3 characters
Maximum entity length	92 characters

4.2 Label Distribution

Label	Count	Percentage
account_number	11,507	11.1%
private_date	11,461	11.0%
private_phone	10,665	10.3%
vkn	10,180	9.8%
secret	10,110	9.7%
private_email	10,000	9.6%
private_address	10,000	9.6%
private_person	10,000	9.6%
iban	10,000	9.6%
tckn	10,000	9.6%

The dataset is intentionally close to balanced across the ten labels. This makes the dataset useful for initial fine-tuning because no single class dominates the training objective. At the same time, the distribution is not necessarily realistic: in real logs or documents, some PII types may appear much more frequently than others.

4.3 Dataset Format

The dataset was published in JSONL format. Each row contains a Turkish text, a span dictionary, and metadata. The spans use Python-style character offsets where the end index is exclusive.

```
{
  "text": "Ahmet Yılmaz için yeni randevu kaydı oluşturuldu.",
  "spans": {
    "private_person: Ahmet Yılmaz": [[0, 12]]
  },
  "info": {
    "id": "synthetic_tr_000001",
    "source": "synthetic_tr"
  }
}
```

For Privacy Filter training, the span dictionary was normalized into the expected label-list structure: each span is represented as a category, start index, and end index.

4.4 Hugging Face Dataset Publication

The dataset was uploaded to Hugging Face as a public dataset under the repository name BTX24/turkish-privacy-pii-ner. Both split files and an all-data file were provided. The split version is intended for standard training/evaluation, while all.jsonl supports custom splits or full-data training.

Hugging Face artifact	URL
Dataset	https://huggingface.co/datasets/BTX24/turkish-privacy-pii-ner
Model	https://huggingface.co/BTX24/turkish-privacy-filter-pii

5. Implementation and Training Pipeline

The model was fine-tuned using the OpenAI Privacy Filter training workflow. A Colab notebook named `privacy_filter_tr_pii_colab.ipynb` was created to make the pipeline reproducible. The notebook covers label-space creation, dataset loading, JSONL normalization, span validation, train/validation/test handling, fine-tuning, evaluation, Hugging Face upload, and post-upload verification.

Training was performed on Google Colab Pro with a GPU runtime. Colab Pro was used because the OpenAI Privacy Filter checkpoint and full synthetic dataset require more stable GPU availability than the free tier typically provides. The project used the OPF-native checkpoint from the original/ subdirectory of `openai/privacy-filter` because the root Hugging Face model directory is intended for a different model format and does not expose the OPF-native encoding field required by the training CLI.

5.1 Training Configuration

Item	Value
Base checkpoint	<code>openai/privacy-filter</code>
Fine-tuned model repository	<code>BTX24/turkish-privacy-filter-pii</code>
Training dataset	<code>BTX24/turkish-privacy-pii-ner</code>
Training examples	83,138
Validation examples	10,392
Test examples	10,393
Best epoch	3
Best selection metric	<code>validation_loss</code>
Best validation loss	0.002157915852249276
Label space version	<code>tr_privacy_v1</code>

5.2 Training Metrics

Epoch	Train loss	Train token accuracy	Validation loss	Validation token accuracy	Elapsed seconds
1	0.038908	0.990768	0.003961	0.999100	1593.69
2	0.002714	0.999463	0.002181	0.999583	1587.53
3	0.001393	0.999704	0.002158	0.999641	1580.02

The model converged rapidly. The largest improvement occurred between epochs 1 and 2, and the best checkpoint was selected at epoch 3. The validation loss decreased only slightly from epoch 2 to epoch 3, suggesting that additional epochs would likely provide limited benefit unless more diverse or real-world training data is added.

5.3 Implementation Problems and Fixes

Several practical problems appeared during the implementation. These problems were important because they directly affected whether the model could be trained, evaluated, and uploaded reproducibly.

Problem	Cause	Fix
JSONDecodeError: Extra data while reading JSONL	The JSONL writer initially wrote multiple JSON objects without newline separators.	The write_jsonl function was fixed to write json.dumps(row) + "\n" for every record. Corrupted split files were regenerated.
Checkpoint config field encoding must be a non-empty string	The root Hugging Face snapshot of openai/privacy-filter did not contain the OPF-native config expected by opf train.	The notebook was changed to download and use the original/ subdirectory from the base model repository.
JSONDecodeError after OPF document prediction	The OPF CLI printed a JSON object followed by additional color legend text on stdout.	The first JSON object was parsed with json.JSONDecoder().raw_decode, and extra non-JSON output was ignored.
Hugging Face push visibility	The first upload workflow allowed private repository creation.	The final upload configuration uses public visibility and exists_ok=True for safer repeated uploads.

These debugging steps were part of the project implementation and helped make the final notebook usable as a complete training and upload workflow.

6. Evaluation Results

The fine-tuned checkpoint was evaluated on the synthetic Turkish test split. The official synthetic test results are very high, but they should be interpreted with caution because the dataset is synthetic and may share generation patterns between splits.

Metric	Value
Test examples	10,393
Test tokens	241,020
Evaluation mode	typed
Loss	0.0028
Token accuracy	0.9996
Inference tokens per second	3027.20

6.1 Detection and Span Metrics

Metric	Value
Detection precision	0.9998
Detection recall	0.9996
Detection F1	0.9997
Detection F2	0.9996
Span precision	0.9988
Span recall	0.9978
Span F1	0.9983
Span F2	0.9980

6.2 Per-Class Span Metrics

Label	Precision	Recall	F1	F2
tckn	1.0000	0.9990	0.9995	0.9992
secret	0.9990	1.0000	0.9995	0.9998
iban	1.0000	1.0000	1.0000	1.0000
vkn	0.9990	0.9990	0.9990	0.9990
account_number	1.0000	0.9992	0.9996	0.9993
private_address	0.9980	0.9940	0.9960	0.9948
private_date	1.0000	1.0000	1.0000	1.0000

private_phone	0.9991	1.0000	0.9995	0.9998
private_email	1.0000	0.9902	0.9951	0.9922
private_person	0.9928	0.9959	0.9943	0.9953

The strongest classes on the synthetic test split were iban and private_date. The relatively weaker classes were private_email, private_person, and private_address. This makes sense because e-mail, person names, and addresses have more boundary variation, suffix interaction, and contextual ambiguity than highly structured numerical identifiers.

7. Qualitative Testing and Error Analysis

In addition to formal evaluation, a longer Turkish document-style test was used to inspect model behavior. This qualitative test included names, TCKN, VKN, IBAN, phone numbers, e-mail addresses, address fragments, and secret-like values in a single paragraph. The test was useful because it revealed a limitation that was not obvious from the synthetic test metrics: the model sometimes confused different numeric or code-like classes when multiple structured identifiers appeared together.

One observed example classified some TCKN, phone, and secret-like spans as iban. It also detected a partial address rather than the complete address span. This shows that although the model performs extremely well on the synthetic held-out split, the synthetic split is not enough to prove production-level robustness. Long, noisy, mixed-entity documents can create class confusions, especially among structured identifiers.

The most important error categories are:

- Boundary errors: the model may capture only part of an address or include/exclude Turkish suffixes incorrectly.
- Type confusion: numeric or code-like strings can be confused across tckn, vkn, iban, account_number, phone, and secret.
- Synthetic-template overfitting: the model may perform better on generated examples than on real-world user text.
- Domain shift: real logs, OCR text, mixed Turkish-English text, or informal spelling can differ substantially from the generated dataset.

8. Discussion

The project demonstrates that OpenAI Privacy Filter can be localized to Turkish through custom label-space fine-tuning and carefully generated synthetic data. The high token accuracy and span F1 indicate that the model learned the synthetic Turkish PII patterns very effectively. The dataset size, label balance, and controlled variation across formats provided a strong foundation for adaptation.

However, this should not be interpreted as a final production anonymization model. The dataset is synthetic. It is excellent for language adaptation, controlled experimentation, and demonstrating that the Privacy Filter training workflow can support Turkish labels. Nevertheless, real-world Turkish privacy text can be much messier. A future version should include manually reviewed real or semi-realistic Turkish examples, domain-specific test sets, and comparison against regex baselines and Turkish BERT-style NER models.

The project is therefore best understood as a strong first Turkish adaptation of an English-centric privacy filter. It shows that the architecture and fine-tuning workflow can be transferred to Turkish. It also identifies the next necessary step: robust evaluation on non-synthetic Turkish data.

9. Ethical Considerations

The dataset was built to avoid real personal data. All examples are synthetic. The project intentionally used fake names, non-routable e-mail domains, mock TCKN/VKN-like values, synthetic phone-number-like strings, artificial IBAN-like values, demo-style secrets, and fake addresses. This design reduces the risk of exposing real PII while still allowing the model to learn privacy-relevant patterns.

The model should not be presented as a KVKK compliance guarantee. It is a privacy assistance tool that can support redaction and data minimization workflows. In high-risk settings, it should be combined with human review, rule-based validation for structured identifiers, auditing, and domain-specific testing.

10. Deliverables

Deliverable	Description
Hugging Face dataset	BTX24/turkish-privacy-pii-ner, containing 103,923 synthetic Turkish PII NER rows.
Hugging Face model	BTX24/turkish-privacy-filter-pii, fine-tuned from openai/privacy-filter.
Training notebook	privacy_filter_tr_pii_colab.ipynb, containing dataset loading, validation, training, evaluation, and upload workflow.
Model artifacts	config.json, model.safetensors, finetune_summary.json, USAGE.txt, label_space.json.
Report	This final project report.

11. Conclusion

This final project successfully created a Turkish privacy-oriented NER dataset, published it on Hugging Face, fine-tuned OpenAI Privacy Filter with a custom Turkish label space, and published the resulting model checkpoint. The model reached 0.9997 detection F1 and 0.9983 span F1 on the synthetic Turkish test split. The project also produced a reproducible Colab workflow and documented practical engineering challenges encountered during dataset normalization, checkpoint selection, OPF training, JSON parsing, and Hugging Face upload.

The final model is significantly better suited for Turkish privacy detection than a purely English-oriented baseline. At the same time, the most important future work is to evaluate and improve the model on manually curated real-world Turkish data. The current model is a strong research and educational artifact, but production usage requires additional validation.

References and Project Links

- [1] OpenAI Privacy Filter announcement. <https://openai.com/index/introducing-openai-privacy-filter/>
- [2] OpenAI Privacy Filter model card. <https://huggingface.co/openai/privacy-filter>
- [3] OpenAI Privacy Filter GitHub repository. <https://github.com/openai/privacy-filter>
- [4] OpenAI Privacy Filter fine-tuning guide. <https://github.com/openai/privacy-filter/blob/main/FINETUNING.md>
- [5] Turkish Privacy PII NER Dataset. <https://huggingface.co/datasets/BTX24/turkish-privacy-pii-ner>
- [6] Turkish Privacy Filter PII model. <https://huggingface.co/BTX24/turkish-privacy-filter-pii>
- [7] Turkish-Wiki-NER-Dataset. <https://github.com/turkish-nlp-suite/Turkish-Wiki-NER-Dataset>
- [8] savasy/bert-base-turkish-ner-cased. <https://huggingface.co/savasy/bert-base-turkish-ner-cased>

Appendix A. Citation

Model citation:

```
@model{toktay_2026_turkish_privacy_filter_pii,  
  title      = {Turkish Privacy Filter PII},  
  author     = {Boran Toktay},  
  year       = {2026},  
  publisher  = {Hugging Face},  
  howpublished = {\url{https://huggingface.co/BTX24/turkish-privacy-filter-pii}},  
  note       = {Fine-tuned OpenAI Privacy Filter checkpoint for Turkish PII span  
detection}  
}
```

Dataset citation:

```
@dataset{toktay_2026_turkish_privacy_pii_ner,  
  title      = {Turkish Privacy PII NER Dataset},  
  author     = {Boran Toktay},  
  year       = {2026},  
  publisher  = {Hugging Face},  
  howpublished = {\url{https://huggingface.co/datasets/BTX24/turkish-privacy-pii-ner}},  
  note       = {Synthetic Turkish privacy-oriented named entity recognition dataset  
for PII detection}  
}
```